



Java Spring Boot Guidelines

Panduan Pembuatan Restful API Menggunakan Java Spring Boot

Riwayat Dokumen

Versi	Tanggal	Penyusun	Ringkasan Perubahan
1.5.0	07-07-2025	Dias Dwi Meidyanto	- Update Mapper remove @Repository - Transaction Management
1.4.0	07-01-2025	Nitha Huwaida Annisa Fatimah Azzahra	- Penambahan terkait koneksi database pada Application Properties - Penggunaan FetchType Lazy untuk Definisikan Relasi ke Tabel Lain
1.3.0	14-06-2024	Dias Dwi Meidyanto	- Controller Example - Anotation Api to Tag - Annotation ApiOperation to Operation - Service Example - Data Response
1.2.0	26-02-2024	Arif Kurniawan	- Spring Boot Version & Vulnerabilities - Collaboration & Deployment - Branching Strategy - Git Tags & Releases - Unit Test
1.1.0	22-02-2024	Dias Dwi Meidyanto	- SonarLint - Caching
1.0.0	20-02-2024	Arif Kurniawan	- Initial Submit

Daftar Isi

Riwayat Dokumen.....	2
Daftar Isi.....	3
Pendahuluan.....	4
Java Spring Boot Guidelines.....	5
1. Packaging.....	5
1.1 Packaging Berdasarkan Tipe Kelas.....	5
1.2 Packaging Berdasarkan Fitur.....	6
2. Auto-Configuration.....	7
3. Dependency Versioning.....	8
4. Application Properties.....	8
5. Setup Swagger.....	9
6. Lombok.....	11
7. Log4j.....	11
8. Spring Boot Actuator.....	12
9. Distribution Management.....	13
10. SonarLint.....	13
11. Caching.....	13
12. Api Design Pattern.....	15
10.1 Error Handling.....	15
10.2 Standar Format Request dan Response.....	20
10.3 Layered System.....	25
10.4 Template Design Pattern.....	30
Collaboration & Deployment.....	33
1. Branching Strategy.....	33
2. Git Tags & Releases.....	35
Unit Testing.....	36
1. Handling Exception.....	36
2. Validation.....	37

Daftar Gambar

Gambar 1. Packaging berdasarkan tipe kelas.....	6
Gambar 2. Packaging berdasarkan Fitur.....	7
Gambar 3. Setup Swagger.....	12
Gambar 4. GitFlow.....	34

Pendahuluan

Petunjuk teknis dalam dokumen ini disusun berdasarkan spesifikasi *dependency* sebagai berikut:

Dependency	Version
Java	8
Spring Boot Starter Parent	2.7.18
My Batis	2.2.2
springdoc-openapi-ui	1.8.0

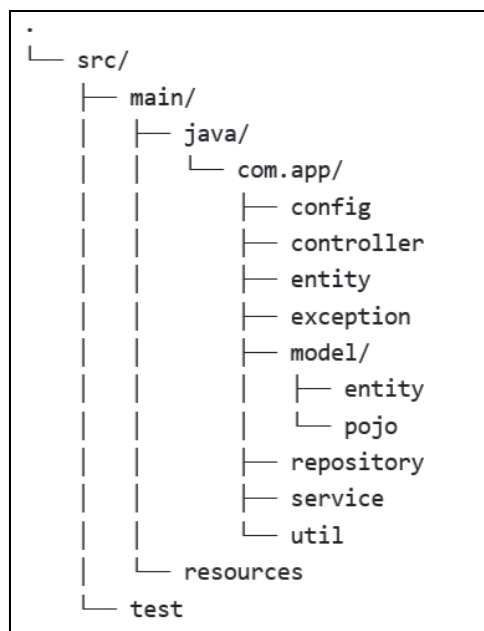
Java Spring Boot Guidelines

1. *Packaging*

Packaging yang tepat akan membantu memahami kode dan alur aplikasi dengan mudah. Setiap *package* memiliki karakteristik yang unik. Berikut adalah *packaging* yang direkomendasikan:

1.1 *Packaging* Berdasarkan Tipe Kelas

Packaging berdasarkan tipe kelas membagi package berdasarkan tipe *Class* dan fungsionalitasnya. *Controller*, *Service*, *Model*, *Repository*, *Configuration*, dan *Utility* lain dikelompokkan dalam Package terpisah. Metode packaging seperti ini cocok diterapkan pada aplikasi berbasis *microservice*. Berikut adalah penerapan *packaging* berdasarkan tipe kelas :



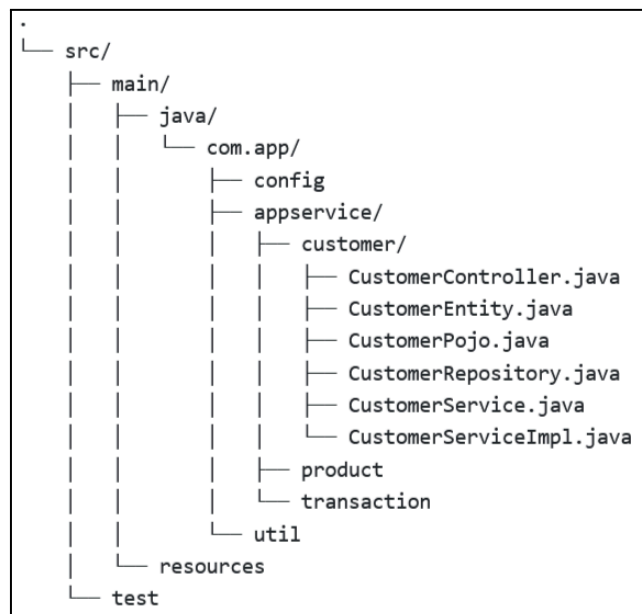
Gambar 1. Packaging berdasarkan tipe kelas

Package	Keterangan
config	Digunakan untuk menyimpan file <i>Class</i> yang berfungsi sebagai konfigurasi aplikasi Java, seperti konfigurasi Database, Swagger, Kafka, dan lain sebagainya.
controller	Digunakan untuk menyimpan file <i>Class</i> <i>@Controller</i> .
service	Digunakan untuk menyimpan file <i>Class</i> <i>@Service</i> .

Package	Keterangan
model	Digunakan untuk menyimpan file <i>Class</i> yang diperuntukkan sebagai <i>Object Model</i> . Dalam <i>package</i> ini file <i>Class</i> dapat dikelompokkan lagi ke dalam beberapa <i>package</i> sesuai fungsionalitasnya. Untuk implementasi JPA, buat <i>package</i> entity untuk menyimpan <i>Class</i> <i>@Entity</i> yang merepresentasikan struktur data pada database dan <i>package</i> pojo untuk menyimpan <i>Class</i> Pojo (<i>Plain Old Java Object</i>) yang berfungsi sebagai <i>object</i> sebuah data.
repository	Digunakan untuk menyimpan file <i>Class</i> <i>@Repository</i> pada implementasi JPA.
mapper	Digunakan untuk menyimpan file <i>Interface Class</i> <i>@Mapper</i> pada implementasi MyBatis.
util	Digunakan untuk menyimpan file <i>Class</i> yang berfungsi sebagai alat bantu dalam membuat suatu proses tertentu dalam program.
exception	Digunakan untuk menyimpan file <i>Class</i> yang berhubungan dengan penanganan <i>Exception</i> seperti <i>@ControllerAdvice</i> .

1.2 Packaging Berdasarkan Fitur

Ketika aplikasi yang dikembangkan memiliki banyak fitur, mengelompokkan file berdasarkan fitur akan memudahkan dalam memahami struktur *Code*. Selain itu keterikatan antara *package* satu dengan *package* lainnya akan sangat rendah. Berikut adalah contoh struktur *packaging* berdasarkan fitur :



Gambar 2. Packaging berdasarkan Fitur

2. *Auto-Configuration*

Salah satu fitur andalan Spring Boot adalah penggunaan Konfigurasi Otomatis. Misalnya jika ingin membangun aplikasi menggunakan JPA untuk akses database, Anda bisa memulainya dengan menyertakan:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Pada contoh diatas, Spring Boot *Auto-Configuration* akan secara otomatis menyesuaikan versi JPA dan *dependencies* lain yang terkait dengan versi Spring Boot yang digunakan. Berikut adalah Spring Boot Starter yang sering digunakan:

Name	Description
spring-boot-starter-thymeleaf	Digunakan untuk membuat aplikasi web menggunakan <i>Thymeleaf</i> views.
spring-boot-starter-data-redis	Digunakan untuk penggunaan Spring Data Redis and the Jedis client.
spring-boot-starter-web	Digunakan untuk membuat aplikasi web, mencakup <i>RESTful applications</i> menggunakan Spring MVC. Ini secara default menyertakan Tomcat pada container.
spring-boot-starter-data-elasticsearch	Digunakan untuk pemrosesan terkait penggunaan Elasticsearch.
spring-boot-starter-test	Digunakan untuk proses <i>Unit Test</i> aplikasi Spring Boot menggunakan library yang mencakup JUnit, Hamcrest, and Mockito.
spring-boot-starter-validation	Digunakan untuk proses validasi menggunakan <i>Hibernate Validator</i> .
spring-boot-starter-data-neo4j	Digunakan untuk Neo4j graph database dan Spring Data Neo4j.
spring-boot-starter-data-jpa	Digunakan untuk Spring Data JPA menggunakan Hibernate.

Name	Description
spring-boot-starter	Digunakan sebagai inti utama, mencakup dukungan auto-configuration, logging, dan YAML.
spring-boot-starter-cache	Digunakan untuk membuat <i>Caching</i> data.
spring-boot-starter-data-rest	It is used for exposing Spring Data repositories over REST using Spring Data REST.
spring-boot-starter-actuator	Digunakan untuk pemantauan aplikasi pada <i>environment production</i> .
spring-boot-starter-log4j2	Digunakan untuk proses <i>logging</i> . Sebagai alternatif untuk spring-boot-starter-logging.
mybatis-spring-boot-starter	Digunakan untuk akses SQL Database.

3. **Dependency Versioning**

Versioning Dependency yang akan digunakan diimplementasikan dengan cara menambahkan properties pada POM.xml seperti contoh berikut :

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.7.18</version>
  <relativePath/>
</parent>
<properties>
  <postgresql.version>42.6.0</postgresql.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>${postgresql.version}</version>
  </dependency>
</dependencies>
```

Versi Spring Boot

Jika menggunakan Java 8, gunakan *Spring Boot* versi 2.7.18 yang tidak memiliki catatan kerentanan (*vulnerabilities*). Sebab *Spring Boot* versi 2.X.X lainnya memiliki catatan kerentanan (*vulnerabilities*).

4. Application Properties

File *properties* digunakan untuk menyimpan sejumlah *property* dalam satu file untuk menjalankan aplikasi di *environment* yang berbeda. Di Spring Boot, properti disimpan dalam file *application.properties* di bawah classpath.

File *application.properties* terletak di direktori `src/main/resources`. Selalu gunakan *Environment Variable* untuk memberikan value pada *property* pada *environment* yang berbeda. Berikut adalah contoh penulisan code pada *application.properties*.

```
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}
spring.datasource.driver-class-name=${SPRING_DATASOURCE_DRIVER}
```

Pada contoh diatas `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`, dan `SPRING_DATASOURCE_DRIVER` adalah *Environment Variable*.

Pada file *application.properties*, perlu ditambahkan konfigurasi standar untuk setiap database yang digunakan pada *service*. Hal ini bertujuan untuk menghindari terjadinya sesi database penuh yang dapat mempengaruhi kinerja sistem secara keseluruhan. Berikut adalah poin-poin konfigurasi yang harus diimplementasikan dalam pembuatan *service* :

- Tambahkan **?ApplicationName=NAMA_SERVICE** setelah url database untuk mempermudah maintenance.

Contoh:
`jdbc:postgresql://pgdev.customs.go.id:5753/teton?ApplicationName=BARKIR_SERVICE_APP`

- Tambahkan nilai default saat melakukan konfigurasi HikariCP

Contoh penulisan default

```
spring.datasource.hikari.maximum-pool-size=${HIKARI_MAXIMUM_POOL_SIZE:10}
```

Nilai default untuk konfigurasi untuk *Single Connection* (PostgreSQL)

```
HIKARI_CONNECTION_TIMEOUT = 10000;
HIKARI_IDLE_TIMEOUT = 60000;
HIKARI_MAX_LIFETIME = 300000;
HIKARI_MAXIMUM_POOL_SIZE = 10;
HIKARI_MINIMUM_IDLE = 1;
```

Nilai default pada **multiple connection**, semua pengaturan harus diseragamkan, terutama pengaturan *max lifetime*, yang sebaiknya disesuaikan dengan nilai terendah.

- Pada konfigurasi *multiple connection*, dianjurkan untuk menonaktifkan konfigurasi default dengan cara sebagai berikut

```
@SpringBootApplication(exclude={DataSourceAutoConfiguration.class})
```

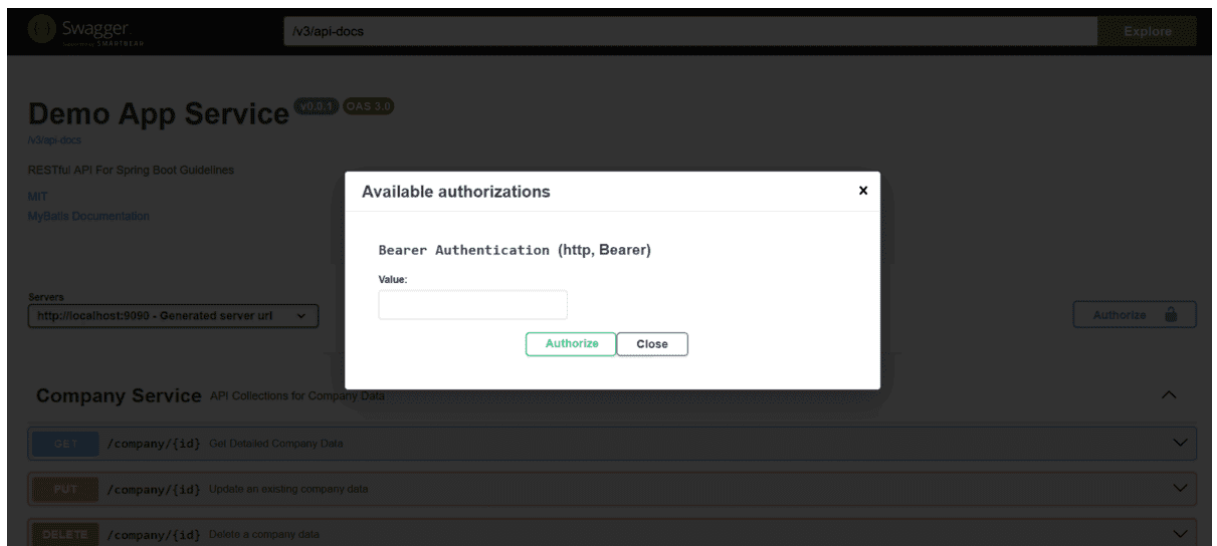
Setelah itu, pengaturan data source perlu didefinisikan atau di setting dengan jelas di direktori konfigurasi servicenya.

5. Setup Swagger

Gunakan *swagger* untuk dokumentasi API yang dibuat. Berikut adalah standar konfigurasi *Swagger* yang direkomendasikan (springdoc versi 1.8.0):

```
@Configuration
@SecurityScheme(
    name = "Bearer Authentication",
    type = SecuritySchemeType.HTTP,
    bearerFormat = "JWT",
    scheme = "bearer"
)
public class SwaggerConfig {
    @Bean
    public OpenAPI springAppOpenAPI() {
        return new OpenAPI()
            .info(new Info().title("Demo App Service")
                .description("RESTful API For Spring Boot
Guidelines")
                .version("v0.0.1")
                .license(new License().name("For Internal Use
Only").url("http://beacukai.go.id")))
            .externalDocs(new ExternalDocumentation()
                .description("Spring Boot Documentation")
                .url("https://bit.ly/4cFhdZi"));
    }
}
```

Dengan konfigurasi diatas Swagger akan menyematkan fitur untuk menambahkan *Header Authorization*. Fitur ini berguna untuk uji coba API yang membutuhkan token dari parameter *Authorization* pada *HTTP Header*.



Gambar 3. Setup Swagger

6. Lombok

Lombok adalah *Java Library* yang dapat digunakan untuk mengurangi kode dan memungkinkan *Developer* menulis kode yang sederhana menggunakan anotasinya.

7. Log4j

Penggunaan *logging* sangat berguna ketika masalah terjadi saat aplikasi dalam *Environment Production*, *logging* adalah satu-satunya cara untuk mengetahui akar permasalahannya. Log4j yang merupakan *framework logging* digunakan untuk mencatatkan *error*, *debug*, info kedalam file .log yang berbeda. Untuk mengaktifkannya, tambahkan dependency berikut pada pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-log4j2</artifactId>
</dependency>
```

Jika menggunakan Log4j sebagai alat untuk proses *logging*, kecualikan *dependency spring-boot-starter-logging* pada *dependency spring-boot-starter*.

Untuk melakukan konfigurasi Log4j, buat file log4j2.xml pada direktori /src/main/resources/. Berikut adalah konfigurasi log4j untuk menghasilkan 3 file log sesuai jenis informasinya:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN" name="SplitLoggingConfig">
  <Appenders>
    <!-- Console Appender -->
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level [%t]
%logger{36} - %msg%n"/>
    </Console>

    <!-- Info Level File Appender -->
    <File name="InfoFile" fileName="logs/app-info.log">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level [%t]
%logger{36} - %msg%n"/>
    </File>

    <!-- Error Level File Appender -->
    <File name="ErrorFile" fileName="logs/app-error.log">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level [%t]
%logger{36} - %msg%n"/>
    </File>

    <!-- Error Level File Appender -->
    <File name="DebugFile" fileName="logs/app-debug.log">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level [%t]
%logger{36} - %msg%n"/>
    </File>
  </Appenders>
  <Loggers>
    <!-- Root Logger -->
    <Root level="info">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="InfoFile" level="info"/>
      <AppenderRef ref="ErrorFile" level="error"/>
      <AppenderRef ref="DebugFile" level="debug"/>
    </Root>
  </Loggers>
</Configuration>
```

8. Spring Boot Actuator

Spring Boot Actuator menyediakan semua fitur dalam *environment production*. *Endpoint actuator* memungkinkan pemantauan aplikasi. Spring Boot menyertakan sejumlah *endpoint* bawaan. Misalnya, *endpoint health* menyediakan informasi *application health*.

Untuk mengaktifkan fitur ini tambahkan *dependency spring-boot-starter-actuator* pada file *pom.xml*.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

9. *Distribution Management*

Gunakan *repository* internal DJBC dengan menambahkan konfigurasi berikut pada POM.xml

```
<distributionManagement>
  <repository>
    <id>bc-2-hosted</id>
    <url>http://mirrorbc.customs.go.id:8081/repository/bc-2-hosted/</url>
  </repository>
  <snapshotRepository>
    <id>bc-2-hosted</id>
    <url>http://mirrorbc.customs.go.id:8081/repository/bc-2-hosted/</url>
  </snapshotRepository>
</distributionManagement>
```

10. *SonarLint*

SonarLint digunakan untuk memastikan *code* yang dibuat memenuhi prinsip *SOLID Programming* dan standar keamanan. *JetBrains Marketplace* menyediakan *plugins SonarLint* yang dapat ditambahkan pada *intellij* atau *code editor* produk *JetBrains* yang lain. Setelah *SonarLint* terinstall dalam *Code Editor*, lakukan binding koneksi ke *SonarCube DJBC*.

11. *Caching*

Caching dapat membantu meningkatkan performa dalam pemrosesan data. Penggunaan *caching query* digunakan pada jenis data yang memiliki frekuensi perubahan, penambahan, dan pengurangan yang sangat minimal. Berikut adalah langkah penerapan *caching query*:

Tambahkan *spring-boot-starter-cache* pada pom.xml.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

Tambahkan anotasi *@EnableCaching* pada *main class* untuk mengaktifkan fitur *caching* bawaan Spring Boot. Berikut adalah penerapannya :

```
@SpringBootApplication
@EnableScheduling
@EnableCaching
public class SinglecoreApplication {
    public static void main(String[] args) {
        SpringApplication.run(SinglecoreApplication.class, args);
    }
}
```

Untuk membuat *caching* data, gunakan anotasi `@Cacheable` pada method *Class* `@Service`. Berikut adalah contoh penerapannya :

```
@RestController
@RefreshScope
@RequestMapping("/v1/kurs")
@CrossOrigin(origins = "*", allowedHeaders = "*")
@Api(description = "Semua API sampling untuk database oracle")
@CacheConfig(cacheNames = "kurs-cache")
public class KursController {

    @Autowired
    KursService kursService;

    @GetMapping(path = "/mon/{kodeValuta}", produces =
MediaType.APPLICATION_JSON_VALUE)
    @ApiOperation(value = "Api Method untuk memonitoring salah satu data
dari TR Kurs")
    @Cacheable(value = "get-kurs-by-kode-valuta-mon", key =
"#kodeValuta")
    public ResponseEntity<?> getOne(@PathVariable("kodeValuta") String
kodeValuta) {

        List<TrKurs> trKurs = kursService.findNilaiKurs(kodeValuta);
        LinkedHashMap<String, Object> res = new LinkedHashMap<>();
        res.put("status", "true");
        res.put("message", "success");
        res.put("data", trKurs);

        return ResponseEntity.ok().body(res);
    }
}
```

Clean cache dapat dilakukan berdasarkan `keyValue` yang telah ditetapkan pada anotasi `@Cacheable`. Berikut adalah contoh penerapan *clean cache*.

```
@RestController
@RequestMapping("/cache")
public class CacheController {

    @Autowired
    public CacheController(CacheManager cacheManager) {
        this.cacheManager = cacheManager;
    }

    private final CacheManager cacheManager;

    @GetMapping("/clear/{cacheName}")
    public ResponseEntity < ? > clearCache(@PathVariable String cacheName) {
        try {
            Cache cache = cacheManager.getCache(cacheName);
            if (cache != null) cache.clear();
            return new ResponseEntity < > ("OK", null, HttpStatus.OK);
        }
    }
}
```

```

        } catch (Exception e) {
            return new ResponseEntity < > ("Error: " + e.getMessage(), null,
HttpStatus.BAD_REQUEST);
        }
    }

    @GetMapping("/clearall")
    public ResponseEntity < ? > clearAll() {
        try {
            cacheManager.getCacheNames().forEach(name -> {
                Cache cache = cacheManager.getCache(name);
                if (cache != null) cache.clear();
            });
            return new ResponseEntity < > ("OK", null, HttpStatus.OK);
        } catch (Exception e) {
            return new ResponseEntity < > ("Error: " + e.getMessage(), null,
HttpStatus.BAD_REQUEST);
        }
    }
}

```

Pada contoh diatas, endpoint `/clear/{cacheName}` berfungsi untuk menghapus *cache* berdasarkan *keyValue* dan endpoint `/clearAll` berfungsi untuk menghapus *cache* secara keseluruhan.

12. **Api Design Pattern**

API Design Patterns adalah aspek penting dari pengembangan perangkat lunak yang memungkinkan pengembang membuat kode yang andal, terukur, dan dapat digunakan kembali. Ini memberikan serangkaian praktik terbaik, spesifikasi, dan standar bersama yang memastikan API dapat diandalkan dan mudah digunakan oleh pengembang lain. Berikut adalah *API Design Patterns* yang perlu diperhatikan:

10.1 *Error Handling*

Error Handling sangat penting dilakukan untuk memudahkan *Client* (Aplikasi *Frontend*) menangani *error* API secara seragam. Misalnya pada aplikasi *React* kita dapat membuat *Error Boundary Component* atau *Error Handling* *Redux* secara terpusat. Berikut adalah penerapan *error handling* pada aplikasi *Spring Boot*:

Customs Default Error Response

Buat sebuah *Class* turunan *Class* *DefaultErrorAttributes* dengan nama *CustomErrorAttributes.java* di dalam *package* ***exception***. *Class* tersebut berfungsi untuk membuat *default response* ketika terjadi *Error*.

```

@Component
public class CustomErrorAttributes extends DefaultErrorAttributes {
    @Override

```



```

        public Map<String, Object> getErrorAttributes(WebRequest
webRequest, ErrorAttributeOptions options) {
            Map<String, Object> errorAttributes =
super.getErrorAttributes(webRequest, options);
            errorAttributes.put("result", "Error");
            errorAttributes.put("detail", errorAttributes.get("error"));
            errorAttributes.put("path", errorAttributes.get("path"));
            errorAttributes.put("date", loggingHolder.getDate());
            errorAttributes.put("code", errorAttributes.get("status"));
            errorAttributes.put("version", loggingHolder.getVersion());
            errorAttributes.remove("status");
            errorAttributes.remove("message");
            errorAttributes.remove("locale");
            errorAttributes.remove("error");
            errorAttributes.remove("cause");
            errorAttributes.remove("trace");
            errorAttributes.remove("timestamp");

            return errorAttributes;

        }
    }
}

```

@ControllerAdvice

Buat sebuah *Class* dengan anotasi *@ControllerAdvice* untuk menghandle beberapa jenis *Exception*. Anotasi ini dapat digunakan untuk menyediakan mekanisme penanganan *Error* secara terpusat untuk seluruh aplikasi yang dijalankan ketika terdapat *Throwing Exception*. Berikut adalah format *response* untuk beberapa jenis *Exception* dan penerapan anotasi *@ControllerAdvice* :

- **DuplicateKeyException**

Terjadi ketika ada pelanggaran data yang bersifat unique pada proses *insert*.

HTTP Code	409 - Conflict
Format Response	{ "status": false, "message": "Data sudah ada" }

- **MethodArgumentNotValidException**

Terjadi ketika data input tidak lolos validasi melalui anotasi *@Valid*.

HTTP Code	422 - Unprocessed Entity
Format Response	{ status: false, message: "Data tidak dapat diproses", errors: ["namaEntitas is required"] }

	}
--	---

- ***NoSuchElementException***

Terjadi ketika data tidak ditemukan di database dalam implementasi JPA. Dapat juga digunakan sebagai standar *exception* terkait tidak ditemukannya suatu data.

HTTP Code	404 – Not Found
Format Response	{ status: false, message: "Data tidak tersedia" }

- ***DataIntegrityViolationException***

Terjadi Ketika data yang berelasi dengan tabel lain tidak valid.

HTTP Code	500 – Internal Server Error
Format Response	{ "status": false, "message": "Data referensi tidak valid!" }

Untuk membuat format response pada setiap jenis *exception* diatas, buat *method* dengan anotasi *@ExceptionHandler* dalam *Class* yang memiliki anotasi *@ControllerAdvice*. Berikut adalah contoh *Class* dengan anotasi *@ControllerAdvice* yang berfungsi untuk menghasilkan *response* untuk beberapa jenis *Exception*:

```
@ControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Object>
    handleValidationErrors(MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult().getFieldErrors()
        .stream().map(FieldError::getDefaultMessage).collect(Collectors.toList
        ());

        HttpStatus status = HttpStatus.UNPROCESSABLE_ENTITY;
        LinkedHashMap<String, Object> response = new
        LinkedHashMap<>();
        response.put("status", "Permintaan tidak dapat diproses.");
        response.put("message", "Data tidak valid!");
        response.put("errors", errors);
        return new ResponseEntity<>(response, new HttpHeaders(),
        status);
    }

    @ExceptionHandler(NoSuchElementException.class)
```

```

        public ResponseEntity<Object>
handleNoSuchElementExceptionErrors(NoSuchElementException ex) {
    HttpStatus status = HttpStatus.NOT_FOUND;
    LinkedHashMap<String, Object> response = new
LinkedHashMap<>();
    response.put("status", "Data tidak tersedia!");
    response.put("message", "Data tidak ditemukan!");
    return new ResponseEntity<>(response, new HttpHeaders(),
status);
}
}

```

Selain Exception bawaan, pengembang juga dapat membuat *Custom Exception* sendiri untuk kebutuhan penanganan *Negative Scenario* seperti data tidak ditemukan, permintaan yang terlarang, dan lain sebagainya. Buat *Class* turunan *RuntimeException* untuk jenis *Exception* baru.

```

public class NotFoundException extends RuntimeException {
    public NotFoundException() {
        super();
    }
    public NotFoundException(String message, Throwable cause) {
        super(message, cause);
    }
    public NotFoundException(String message) {
        super(message);
    }
    public NotFoundException(Throwable cause) {
        super(cause);
    }
}

```

Tambahkan *method* *@ExceptionHandler* pada *Class* dengan anotasi *@ControllerAdvice* untuk jenis *Exception* yang baru ditambahkan.

```

@ExceptionHandler(NotFoundException.class)
public ResponseEntity < Object >
handleForbiddenExceptionErrors(NotFoundException ex) {
    HttpStatus status = HttpStatus.NOT_FOUND;
    LinkedHashMap < String, Object > response = new LinkedHashMap < >
();
    response.put("status", false);
    response.put("message", "Data tidak ditemukan!");
    return new ResponseEntity < > (response, new HttpHeaders(),
status);
}

```

Setelah `@ControllerAdvice` diimplementasikan, pengembang tidak perlu lagi membuat `code API Response` untuk menangani setiap *Exception* atau *Negative Scenario* penggunaan seperti data tidak ditemukan dan sebagainya. Untuk memicu `@ControllerAdvice` bekerja cukup lakukan dengan *throwing Exception*. Berikut adalah contoh penerapannya:

```
public ArtikelHukumPojo getById(UUID id) {
    try {
        Optional<ArtikelHukum> data =
artikelHukumRepository.findById(id);
        if (data.isPresent()) {
            BigInteger countViewArtikel =
viewArtikelRepository.countViewArtikel(id);
            ArtikelHukumPojo result = toPojo(data.get());
            result.setViewCount(countViewArtikel);
            return result;
        } else {
            throw new NotFoundException("Data tidak ditemukan");
        }
    } catch (Exception e) {
        log.error("Error when get a artikel.", e);
        throw e;
    }
}
```

10.2 Standar Format *Request* dan *Response*

GET Datatable

Use Case	Digunakan pada datatable aplikasi <i>Frontend</i> .
Endpoint	/CONTROLLER_PATH/
Method	GET
Query Param	?page=1&limit=10&filterParam1=&filterParam2
HTTP Code	200
Response	<pre>{ result: "Success", detail: "Data(s) successfully fetched.", date: "03-05-2024 09:37:56:40 WIB", path: "/barang", code: 200, version: "1.0.0", data: { page: 1, limit: 10, total: 182, list:[{ idEntitas: "2834uo-auu3929fe-e8ed3920", namaEntitas: "PT Sempat Jaya Tbk", nibEntitas: "PT Sempat Jaya Tbk", waktuRekam: "2024-02-12 11:12:05" }] } }</pre>

GET All Data

Use Case	Digunakan pada <i>combo box</i> dan sejenisnya pada aplikasi <i>Frontend</i> .
Endpoint	/CONTROLLER_PATH/findAll
Method	GET
HTTP Code	200

<i>Response</i>	<pre> { result: "Success", detail: "Data(s) successfully fetched.", date: "03-05-2024 09:37:56:40 WIB", path: "/entitas", code: 200, version: "1.0.0", data:[{ idEntitas: "2834uo-auu3929fe-e8ed3920", namaEntitas: "PT Sempat Jaya Tbk", nibEntitas: "PT Sempat Jaya Tbk", waktuRekam: "2024-02-12 11:12:05" }] } </pre>
-----------------	---

GET One Data

<i>Use Case</i>	Digunakan pada halaman detil data
<i>Endpoint</i>	/CONTROLLER_PATH/find/
<i>Method</i>	GET
<i>Query Param</i>	?id={DATA_KEY}
<i>HTTP Code</i>	200
<i>Response</i>	<pre> { result: "Success", detail: "Data(s) successfully fetched.", date: "03-05-2024 09:37:56:40 WIB", path: "/entitas/findOne", code: 200, version: "1.0.0", data: { idEntitas: "2834uo-auu3929fe-e8ed3920", namaEntitas: "PT Sempat Jaya Tbk", nibEntitas: "PT Sempat Jaya Tbk", waktuRekam: "2024-02-12 11:12:05" } } </pre>

Create

<i>Use Case</i>	Digunakan untuk menyimpan data baru.
<i>Endpoint</i>	/CONTROLLER_PATH/
<i>Method</i>	POST
<i>HTTP Code</i>	200

<i>Request Body</i>	<pre>{ namaEntitas: "PT Sempat Jaya Tbk", nibEntitas: "PT Sempat Jaya Tbk" }</pre> <i>Note: ID, waktu rekam, dan nib rekam tidak disertakan (Diisi secara otomatis).</i>
<i>Response</i>	<pre>{ result: "Success", detail: "Data(s) successfully created.", date: "03-05-2024 09:37:56:40 WIB", path: "/entitas", code: 200, version: "1.0.0", data: { idEntitas: "2834uo-auu3929fe-e8ed3920", namaEntitas: "PT Sempat Jaya Tbk", nibEntitas: "PT Sempat Jaya Tbk", waktuRekam: "2024-02-12 11:12:05" } }</pre>

Update

<i>Use Case</i>	Digunakan untuk mengubah data.
<i>Endpoint</i>	/CONTROLLER_PATH/
<i>Query Param</i>	?id={DATA_KEY}
<i>Method</i>	PUT
<i>Request Body</i>	<pre>{ namaEntitas: "PT Sempat Jaya Tbk", nibEntitas: "PT Sempat Jaya Tbk" }</pre> <i>Note: Waktu update tidak disertakan (Diisi secara otomatis).</i>
<i>HTTP Code</i>	200
<i>Response</i>	<pre>{ result: "Success", detail: "Data(s) successfully modified.", date: "03-05-2024 09:37:56:40 WIB", path: "/entitas", code: 200, version: "1.0.0", data: { idEntitas: "2834uo-auu3929fe-e8ed3920", namaEntitas: "PT Sempat Jaya Tbk", nibEntitas: "PT Sempat Jaya Tbk", waktuRekam: "2024-02-12 11:12:05" } }</pre>

Delete

<i>Use Case</i>	Digunakan untuk menghapus data.
<i>Endpoint</i>	/CONTROLLER_PATH/
<i>Method</i>	DELETE
<i>Query Param</i>	?id={DATA_KEY}
<i>HTTP Code</i>	200
<i>Response</i>	<pre>{ result: "Success", detail: "Data(s) successfully deleted.", date: "03-05-2024 09:37:56:40 WIB", path: "/entitas", code: 200, version: "1.0.0" }</pre>

Berikut adalah *Class-Class* yang digunakan sebagai format standar *response*.

Datatable Response

```
@Setter
@Getter
@JsonInclude(JsonInclude.Include.NON_NULL)
public class DatatableResponse<T> {
    String result = "Success";
    String detail = "Data fetched";
    String path = "/barang";
    String date =
    LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy
    HH:mm:ss:SS 'WIB'"));
    int code = 200;
    String version = "1.0.0";
    PageDataResponse<T> data;
}
```

All Data Response

```
@Setter
@Getter
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ListResponse<T> {
    private String status;
    private String message;
    private List<T> data;

    public static <T> ListResponse<T> success(String status, String
    message, List<T> data) {
        ListResponse<T> response = new ListResponse<>();
        response.setStatus(status);
        response.setMessage(message);
        response.setData(data);
        return response;
    }
}
```

Data Response

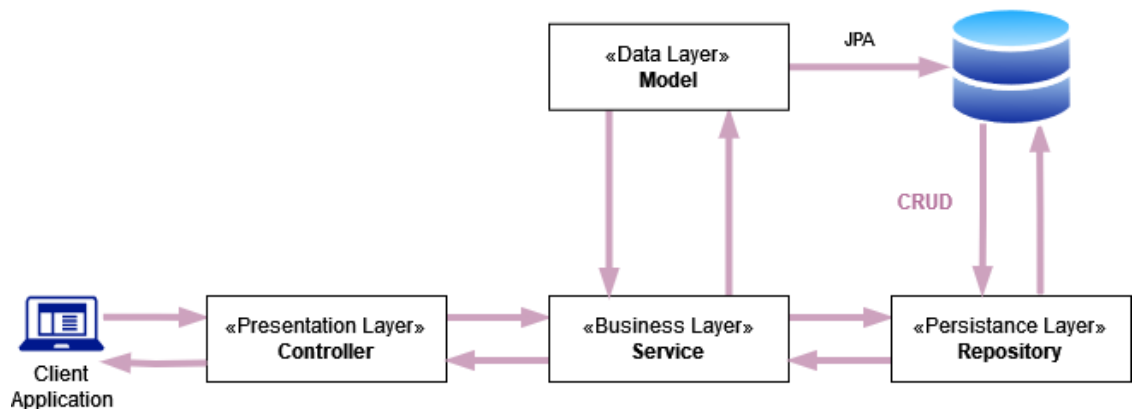
```
@Setter
@Getter
@JsonInclude(JsonInclude.Include.NON_NULL)
public class DataResponse<T> {
    String result = "Success";
    String detail = "Data fetched";
    String path = "/barang";
    String date =
    LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy
    HH:mm:ss:SS 'WIB'"));
    int code = 200;
    String version = "1.0.0";
    T data;
}
```

Default Response

```
@Setter
@Getter
public class DefaultReponse {
    String result = "Success";
    String detail = "Data fetched";
    String path = "/barang";
    String date =
    LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy
    HH:mm:ss:SS 'WIB'"));
    int code = 200;
    String version = "1.0.0";
}
```

10.3 Layered System

RESTful API harus dirancang dengan sistem berlapis, di mana setiap lapisan menyediakan serangkaian fungsi tertentu. Hal ini memungkinkan fleksibilitas yang lebih besar dalam sistem, karena dalam setiap lapisan dapat ditambahkan, dihapus, atau dimodifikasi tanpa mempengaruhi keseluruhan sistem :



Entity (Penggunaan JPA)

Entity adalah sebuah *object* data yang merepresentasikan *attribute* dari tabel. Berikut contoh pembuatan *entity*:

```
@Getter
@Setter
@Entity
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "data_entitas")
public class DataEntitas {

    @Id
    @Column(name = "id_entitas")
    private String idEntitas;
    @Column(name = "id_header")
    private String idHeader;
    @Column(name = "nama_entitas")
    private String namaEntitas;
    @Column(name = "nib_entitas")
    private String nibEntitas;
    @Column(name = "nip_rekam")
    private String nipRekam;
    @Column(name = "waktu_rekam")
    private Timestamp waktuRekam;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name="kode_entitas", referencedColumnName="kode_entitas")
    private TrEntitas refEntitas;
}
```

Pada contoh diatas cukup gunakan anotasi *@Getter* dan *@Setter* daripada *@Data* jika tidak ada kebutuhan penggunaan anotasi *@ToString*, *@EqualsAndHashCode*, dan *@RequiredArgsConstructor*.

Penggunaan *@JsonProperty(access = JsonProperty.Access.READ_ONLY)* bertujuan agar *property* tersebut tidak muncul pada proses POST / PUT.

Disarankan untuk menggunakan **FetchType.LAZY** ketika mendefinisikan relasi ke tabel lain agar table child atau parent yang tidak diperlukan tidak ikut ter-*fetch*.

Pojo

Pojo atau *DTO* berfungsi menyediakan atribut data yang digunakan oleh *Service* , *Controller*, dst. Untuk kasus penggunaan sebagai data input, *Pojo* atau *DTO* hendaknya didefinisikan spesifikasi atribut untuk kebutuhan validasi dan juga dokumentasi Swagger seperti contoh berikut:

```
@Getter
@Setter
public class DataEntitas {
    @NotNull(message = "idHeader harus disertakan")
    @Empty(message = "idHeader tidak boleh kosong")
    private String idHeader;
    private String idEntitas;
    @NotNull(message = "namaEntitas harus disertakan")
    @Empty(message = "namaEntitas tidak boleh kosong")
    @Size(max = 200, message = "namaEntitas maksimal 200 karakter")
    private String namaEntitas;
    @NotNull(message = "nibEntitas harus disertakan")
    @Empty(message = "nibEntitas tidak boleh kosong")
    @Size(max = 20, message = "nibEntitas maksimal 20 karakter")
    private String nibEntitas;
    @JsonProperty(access = JsonProperty.Access.READ_ONLY)
    private String nipRekam;
    @JsonProperty(access = JsonProperty.Access.READ_ONLY)
    private Timestamp waktuRekam;
}
```

Repository (Kasus Penggunaan JPA)

Repository berfungsi untuk memproses data CRUD ke Database. Dalam kasus penggunaan JPA, poin-poin berikut perlu diperhatikan.

- Untuk proses SELECT atau pengambilan data, gunakan *Interface-based projections* agar dapat disesuaikan dengan kebutuhan. Misalnya untuk mengambil data *column* *idEntitas* dan *namaEntitas* dari tabel data Entitas yang memiliki kolom sangat banyak. Berikut adalah contoh implementasi proses SELECT menggunakan *Interface* pada JPA.

Interface

```
public interface EntitasAll {
    String getIdEntitas();
    String getNamaEntitas();
}
```

Repository

```
@Repository
public interface DataEntitasRepository extends
JpaRepository<DataEntitas, String> {
    @Query(value = "SELECT id_entitas idEntitas, nama_entitas
namaEntitas FROM td_entitas", nativeQuery = true)
    List<EntitasAll> findAllItem();
}
```

- Direkomendasikan untuk menggunakan *JPA Specification* daripada *Native Query* untuk kasus penggunaan permintaan data dengan *paging* dan *ordering*. Sebab *Native Query* tidak mendukung Parameter *Order*.

Mapper (Kasus Penggunaan MyBatis)

Mapper Interface berfungsi untuk memproses data CRUD ke Database. Berikut adalah contoh Pembuatan Mapper:

```
@Mapper
public interface EntitasMapper {
    List<DataEntitas> getDataEntitas(@Param("idHeader") String
idHeader);
}
```

Mapper XML berisi *Query* yang digunakan untuk mendapatkan informasi dalam database. Letakkan file mapper.xml pada direktori *resources*. Berikut adalah contoh isi dari *mapper xml*:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd" >
<mapper namespace="id.go.beacukai.singlecore.mapper.EntitasMapper">
    <select id="getDataEntitas"
resultType="id.go.beacukai.singlecore.model.DataEntitas">
        SELECT A.NAMA_ENTITAS, A.NIB_ENTITAS
        FROM PFPD.TD_ENTITAS
        WHERE A.ID_HEADER = #{idHeader}
    </select>
</mapper>
```

Service

Service berfungsi sebagai *Business Logic* yang bertugas memproses suatu permintaan yang diteruskan oleh *Controller* dan memberikan *output* atas permintaan tersebut.

Berikut adalah contoh pembuatan method pada *service*.

```
@Override
    public DataResponse<EntitasPojo> create(EntitasPojo entitas) {
        try {
            entitas.setNipRekam(headerHolder.getNip());
            entitas.setWaktuRekam(new DateHelper().getCurrentTimestamp());
            entitas.setIdEntitas(UUID.randomUUID().toString());
            entitasMapper.insertEntitas(entitas);
            EntitasPojo data =
            entitasMapper.getDetailEntitas(entitas.getIdEntitas());
            return new DataResponse<>(SUCCESS, ResponseMessage.DATA_CREATED,
            loggingHolder.getPath(), loggingHolder.getDate(), HttpStatus.OK.value()
            , loggingHolder.getVersion(), data);
        } catch (Exception e) {
            log.error("Error when create a entitas.", e);
            throw e;
        }
    }
}
```

Berikut adalah poin-poin yang harus digunakan dalam pembuatan *service* :

1. Selalu gunakan *Try and Catch*
2. Terapkan *Logging Error* Menggunakan *Log4j* pada block *Catch*
3. Terapkan *Throwing Exception* untuk *Negative Scenario* (Error, Data Tidak Ditemukan, dll)
4. Return ke Format *Response Model* yang sudah Ditentukan. Format response model akan membantu *frontend* untuk mengolah data.
5. Menggunakan Transaction

Penggunaan mekanisme transaksi pada service ini diperlukan untuk memastikan konsistensi data. Apabila terjadi kesalahan (error/exception) di tengah proses, maka seluruh operasi penyimpanan atau pembaruan data yang telah dilakukan sebelumnya akan dibatalkan (rollback). Dengan demikian, sistem terhindar dari kondisi di mana sebagian data berhasil diubah sementara sebagian lainnya tidak, yang dapat menyebabkan inkonsistensi atau kerusakan data.

- Transactional (JPA)

```
@Override
@Transactional
public DataResponse<EntitasPojo> create(EntitasPojo entitas) {
    try {
        entitas.setNipRekam(headerHolder.getNip());
        entitas.setWaktuRekam(new DateHelper().getCurrentTimestamp());
        entitas.setIdEntitas(UUID.randomUUID().toString());
        entitasMapper.insertEntitas(entitas);
        EntitasPojo data =
        entitasMapper.getDetailEntitas(entitas.getIdEntitas());
        return new DataResponse<>(SUCCESS, ResponseMessage.DATA_CREATED,
        loggingHolder.getPath(), loggingHolder.getDate(), HttpStatus.OK.value()
        , loggingHolder.getVersion(), data);
    } catch (Exception e) {
        log.error("Error when create a entitas.", e);
        TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()
        ;
        throw e;
    }
}
```

- Transaction (My Batis)

```
@Override
public void saveCNJson(List<CNRequestDTO> datas, List<String>
dataKafka, List<String> dataKafkaPerbaikan) {
```

```

        TransactionStatus txStatus =
transactionManager.getTransaction(new
DefaultTransactionDefinition());
        try {
            datas.forEach(data -> {
                fobTotal = 0;
                Header header = new Header();

                header =
cnMapper.getDataHeader(data.getNomorBarangKiriman(),
yyyyMMdd.format(data.getTanggalBarangKiriman()));
                data.setNomorAju(header == null ? new
NomorAjuHelper().getNomorAju(data.getJenisAju(), data.getKantor()) :
header.getNomorAju());

data.setNomorDaftar(header==null?null:header.getNomorDaftar());

data.setNamaKantor(trKantorMapper.getNamaKantorPendek(data.getKantor(
)));

data.setNamaGudang(trGudangMapper.getNamaGudang(data.getTps(),
data.getKantor()));

data.setNdpbm(trKursMapper.getNilaiKurs(data.getValuta()));

data.setNamaNegaraTujuan(trNegaraMapper.getNamaNegara(data.getNegaraT
ujuan()));

data.setNamaDaerahAsal(trDaerahAsalMapper.getNamaDaerahAsal(data.getDa
erahAsalBarang()));

data.setNamaJenisEkspor(trJenisEksporMapper.getNamaJenisEkspor(data.g
etJenisEkspor()));

                if ((data.getCnBarangList() == null ||
data.getCnBarangList().isEmpty()) &&
!(data.getJenisAju()+"").equals("13"))
                    throw new IllegalArgumentException("Data Barang
untuk " + data.getNomorBarangKiriman() + " harus diisi");
                if ((data.getCnSuratList() == null ||
data.getCnSuratList().isEmpty()) &&
(data.getJenisAju()+"").equals("13"))
                    throw new IllegalArgumentException("Detail Daftar
Barang Kiriman untuk " + data.getNomorBarangKiriman() + " harus
diisi");

                processingDataList(data);
                data.setJenisAju(data.getJenisAju() == null ?
"11":data.getJenisAju());

                int cekNoBarang =
cnMapper.cekTdHeaderNoBarangAndNoDaftarNotNull(data.getNomorBarangKir
iman(), data.getJenisAju());
                if(data.getJenisAju().equals("11") ||
data.getJenisAju().equals("14")){
                    insertCN11(data, header, cekNoBarang, dataKafka,
dataKafkaPerbaikan);
                }
            });
        }
    }
}

```



```

        }else {
            insertCN13(data);
        }
    });
    transactionManager.commit(txStatus);
} catch (Exception e) {
    transactionManager.rollback(txStatus);
    log.error("Gagal save CN ", e);
    throw e;
}
}

```

Controller

Controller berfungsi sebagai Routing Restful API yang merupakan target akhir dari permintaan. Kemudian permintaan akan diserahkan ke layer Service untuk diproses. Berikut adalah contoh pembuatan method pada controller :

```

@RestController
@RequestMapping("/entitas")
@Tag(name = "Entitas Service", description = "API Collections for Entitas Data")
public class EntitasController {
    private final EntitasService entitasService;

    public EntitasController(EntitasService entitasService) {
        this.entitasService = entitasService;
    }

    @PostMapping(produces = MediaType.APPLICATION_JSON_VALUE)
    @Operation(
        summary = "Create a new entitas",
        description = "Create a new entitas and save it into the data source"
    )
    public ResponseEntity<DataResponse<EntitasPojo>> create(@Valid
    @RequestBody EntitasPojo entitas) {
        DataResponse<EntitasPojo> data =
        entitasService.create(entitas);
        return ResponseEntity.ok().body(data);
    }
}

```

Berikut adalah poin-poin yang harus ada pada kelas *controller* :

- Anotasi *@Tag*
@Tag digunakan untuk pengelompokan endpoint ke dalam kategori tertentu untuk tujuan organisasi dan dokumentasi.

- Anotasi *@Operation*
Anotasi *@Operation* berguna untuk memberikan deskripsi di setiap endpoint pada dokumentasi *swagger*.
- Penggunaan Anotasi *@Valid*
Anotasi *@Valid* berguna untuk melakukan validasi *Request Body* terkait dengan peraturan yang sudah dibuat pada *model* , *pojo* ataupun *dto*.

10.4 Template Design Pattern

Template Design Patterns merupakan desain satu atau lebih method yang digunakan berulang kali yang bisa digunakan pada beberapa method yang akan digunakan. Misalnya pembuatan method *JpaSpecification* yang dapat digunakan untuk *filtering* data ataupun pembuatan method *HeaderInterceptor* berfungsi untuk mendapatkan *value* dari hasil ekstraksi *token*.

Template *Design Pattern* dapat dilihat pada *repository* berikut:

JPA

<https://gitlab.customs.go.id/arifkurn/springboot-demo-guidelines/tree/jpa-guidelines>

MyBatis

<https://gitlab.customs.go.id/arifkurn/springboot-demo-guidelines/tree/mybatis-guidelines>

Berikut adalah contoh penerapan *template design pattern* :

HeaderHolder

Pembuatan *Utility header interceptor* yang berfungsi untuk *mengextract* informasi dari *token*. Berikut adalah contoh pembuatan serta penggunaannya dalam *Service*:

Pembuatan JwtUtil :

```
public class JwtUtil {
    private static final Logger logger =
    LoggerFactory.getLogger(JwtUtil.class);

    public static final String DEFAULT_TOKEN_PREFIX = "Bearer ";
    private JwtUtil() {
        throw new UnsupportedOperationException("This is a utility
        class and cannot be instantiated");
    }

    public static String extractAuthToken(String authHeader, String
    prefix){
```

```

        String token = Optional.ofNullable(authHeader).orElse("");
        if (prefix != null && token.startsWith(prefix)){
            return token.replaceFirst(prefix, "");
        }
        return token;
    }

    public static Map<String, String> getValueFromToken(String token,
String[] attrName){
        if (StringUtils.isEmpty(token)) return new HashMap<>();

        JSONParser jsonParser = new JSONParser();
        String[] chunks = token.split("\\.");
        String payload = chunks.length > 1 ? chunks[1]:chunks[0];
        try {
            JSONObject obj = (JSONObject) jsonParser.parse(new
String(Base64.getDecoder().decode(payload)));
            Map<String, String> result = new HashMap<>();
            for (String s : attrName) {
                System.out.println(s);
                result.put(s, (String) obj.getDefault(s, ""));
            }
            return result;
        } catch (ParseException e) {
            logger.error("Failed to extract value from token!", e);
        }
        return new HashMap<>();
    }
}

```

Pembuatan *HeaderInterceptorConfig* :

```

@Configuration
public class HeaderInterceptorConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors(final InterceptorRegistry registry) {
        registry.addInterceptor(headerInterceptor());
    }

    @Bean
    public HeaderInterceptor headerInterceptor() {
        return new HeaderInterceptor(headerHolder());
    }

    @Bean
    @Scope(value = WebApplicationContext.SCOPE_REQUEST, proxyMode =
ScopedProxyMode.TARGET_CLASS)
    public HeaderHolder headerHolder() {
        return new HeaderHolder();
    }
}

```

Pembuatan *HeaderInterceptor* :

```

public class HeaderInterceptor implements HandlerInterceptor {

    private final HeaderHolder headerHolder;

    @Override
    public boolean preHandle(HttpServletRequest request,
        HttpServletResponse response, Object handler) throws Exception {
        String token = request.getHeader("Authorization");
        Map<String, String> tokenValue =
        JwtUtil.getValueFromToken(token, new String[]{"nip", "name",
        "given_name", "preferred_username", "role", "kode_kantor"});
        String nip = tokenValue.getOrDefault("nip", "");
        headerHolder.setNip(nip);
        return true;
    }

    public HeaderInterceptor(HeaderHolder headerHolder) {
        this.headerHolder = headerHolder;
    }

}

```

Pembuatan HeaderHolder :

```

@Data
public class HeaderHolder {
    private String nip;
}

```

Penggunaan HeaderHolder :

```

@Service("artikelService")
public class ArtikelHukumServiceImpl implements ArtikelHukumService {
    @Autowired
    private HeaderHolder headerHolder;

    private static final Logger log =
    LogManager.getLogger(ApiExceptionHandler.class);

    @Autowired
    private ArtikelHukumRepository artikelHukumRepository;

    @Override
    public ArtikelHukumPojo save(ArtikelHukumPojo data) {
        try {
            ArtikelHukum dt = new ArtikelHukum();
            dt.setIdArtikelHukum(UUID.randomUUID());
            dt.setNipRekam(headerHolder.getNip());
            dt.setWaktuRekam(new Util().getCurrentTimestamp());
            ArtikelHukum dataInsert = toEntity(data, dt);
            artikelHukumRepository.save(dataInsert);
            ArtikelHukumPojo result = toPojo(dataInsert);
            eventLogger.LogActivity("Create", "Artikel Hukum",
            result);
            return result;
        } catch (Exception e) {
            log.error("Error when create a artikel.", e);
            throw e;
        }
    }
}

```

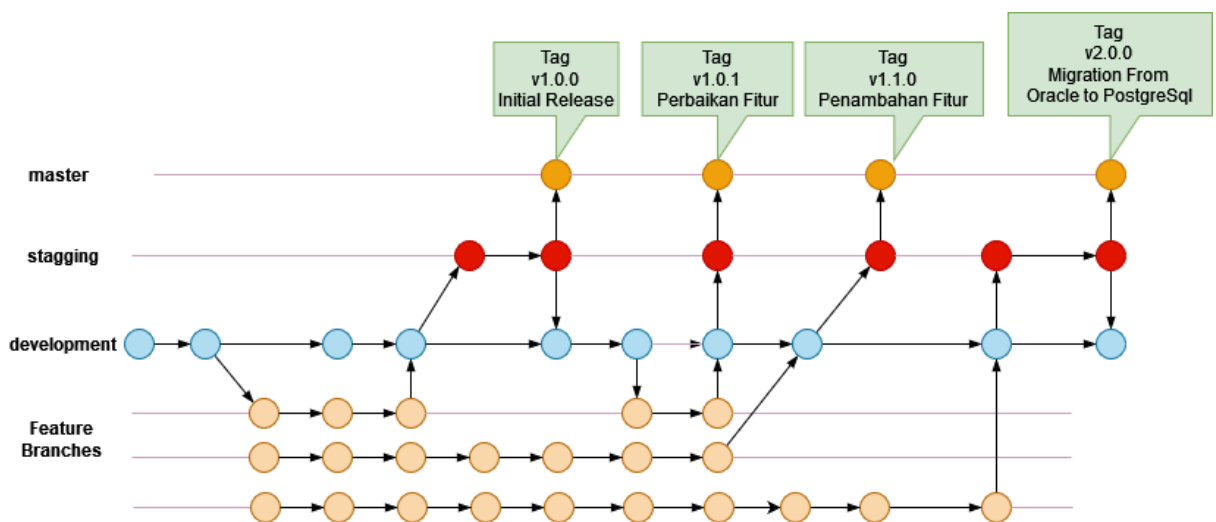
```
}  
  }  
}
```

Collaboration & Deployment

Dalam lingkungan pengembangan perangkat lunak saat ini, semakin umum bagi banyak pengembang untuk berkolaborasi dalam proyek yang sama secara bersamaan. Mengelola perubahan kode secara efisien, menyelesaikan konflik, dan mempertahankan basis kode yang koheren menjadi aspek penting dari kolaborasi yang sukses. Berikut adalah strategi pengelolaan *code* yang harus diperhatikan:

1. Branching Strategy

Secara umum beberapa *branch* dibuat untuk mengakomodir beberapa *environment* yang diimplementasikan, seperti *environment production*, *staging*, dan *development*. Berikut adalah GitFlow yang digunakan:



Gambar 4. GitFlow

■ Master

Branch *master* diperuntukkan bagi *environment production*. Aktivitas yang terjadi pada branch ini hanya aktivitas *merge* dari *branch staging*. *Merge* dilakukan ketika aplikasi yang dikembangkan telah disetujui oleh QA pada *environment Staging*. Setelah *merge* dilakukan berikan *Git Tags* versi rilis.

```
$ git checkout master
Switched to branch 'master'
$ git merge --no-ff staging
Merge made by recursive.
(Summary of changes)
$ git tag -a 1.0.0 -m "Initial Release" -n "Bug Fix datatable RestApi"
```

■ Staging

Branch *staging* digunakan untuk mempersiapkan rilis aplikasi. Aktivitas yang terjadi pada branch ini mencakup beberapa hal sebagai berikut:

- Merge dari *branch development*.
- Uji performa.
- *Vulnerability Analysis*.
- Setiap perubahan yang dilakukan pada branch ini harus dimerge kembali ke *branch development*.

■ Development

Branch *development* merupakan titik awal dari seluruh *branch* yang ada. Aktivitas yang terjadi pada branch ini mencakup beberapa hal sebagai berikut:

- Merge dari *feature branches*.
- Uji fungsionalitas.
- *Code Review* menggunakan *SonarCube* pada *pipeline* dan atau secara review manual.
- Resolve Conflict.
- Merge setiap perubahan code pada *branch staging* ke *branch development*.

Branch ini dapat disebut sebagai *branch* tempat integrasi code antar pengembang dilakukan.

Dalam kondisi tertentu ⁽¹⁾ dan jika memungkinkan, berikan *role developer* pada akun GIT pengembang dan cegah mereka melakukan merging ke *branch development* secara mandiri. Proses *merging* dilakukan dengan cara mengajukan permintaan *merge* (*Request Merge*) ke *branch development*. Selanjutnya permintaan tersebut akan divalidasi oleh *Maintener* yang diperankan oleh *Technical Lead* atau pengembang senior atau penanggung jawab utama dari aplikasi yang sedang dikembangkan. Hal ini ditujukan untuk menjaga kualitas *code* dan penanganan yang baik jika terjadi *conflict*.

■ Feature Branches

Untuk setiap aktivitas pengembangan (bug, peningkatan, dll), *branch* baru dibuat dari *branch development*. Pengembang tidak harus bekerja di *branch* yang sama, karena setiap *branch* fitur hanya mencakup apa yang sedang dikerjakan oleh pengembang tunggal tersebut. Di sinilah *branching GIT* berguna.

Setelah fitur yang dikembangkan oleh pengembang tunggal tersebut siap, fitur tersebut digabungkan kembali (*Merge*) secara lokal ke dalam *branch development*. Yang perlu diperhatikan, setiap pengembang diwajibkan untuk melakukan proses *pulling branch development* ke *branch* lokal mereka agar tetap setiap kali akan melakukan aktivitas pengembangan *source code* lokal pengembang selalu sinkron dengan *source code* aplikasi terbaru.

¹ Misalnya ketika terjadi *conflict* dan pengembang masih junior atau pemula, potensi terjadinya masalah pada saat *resolving conflict* semakin besar.

2. Git Tags & Releases

Saat mempersiapkan rilis, gunakan *Git Tags dan Releases*. Tag memberikan nama atau nomor versi yang bermakna untuk menandai titik tertentu dalam sejarah repositori. Rilis di platform hosting Git memungkinkan distribusi versi yang stabil dengan mudah. Dengan menggunakan tag dan rilis, tim dapat melacak kemajuan proyek dan dengan mudah mengidentifikasi versi stabil untuk penerapan.

Berikut langkah-langkah dalam membuat *Git Tags & Releases* melalui Gitlab UI.

1. Buka menu **Repository > Tags**
2. Klik tombol **New Tags**
3. Berikan nama Tag.
4. Untuk isian “Create From”, pilih branch **Master**.
5. Tambahkan pesan dan atau catatan rilis.
6. Klik tombol “Create Tag”.

Contoh GIT Tags & Releases

v1.0.0

9d76f8ac v1.0.0 • released 12 hours ago by 

Assets 4

 [Source code](#) ▾

Release version v1.0.0

★ Feature

- Springboot Auto-Configuration
- Layered System
- CRUD Process
- Pagination
- Dynamic Ordering
- Standard HTTP Request & Response
- Custom Error Attributes
- Converting Entity to Other Data Model
- Exception Handler
- HTTP Header Interceptor
- Unit Test

<https://gitlab.customs.go.id/arifkurn/springboot-demo-guidelines/-/releases>

Unit Testing

Unit Testing adalah proses di mana Anda menguji unit fungsional terkecil dari kode. Pengujian perangkat lunak membantu memastikan kualitas kode, dan ini merupakan bagian yang tidak dapat terpisahkan dari pengembangan perangkat lunak. Berikut ini adalah beberapa kasus penggunaan *Unit Testing*:

1. Handling Exception

Sebagaimana telah dijelaskan pada BAB sebelumnya, perangkat lunak yang baik memiliki *Design Pattern* yang dijadikan standar pengembangan. Salah satunya adalah penanganan error atau negatif skenario yang seragam. Untuk memastikan code memenuhi standar penanganan error atau negatif skenario tersebut *Unit Testing* dapat digunakan untuk pengujian.

DATA NOT FOUND

Method pada *Class* `EntitasServiceImpl.java`.

```
@Override
public DataResponse < EntitasPojo > findOne(String idEntitas) {
    try {
        Optional < TdEntitas > data = entitasRepository.findById(idEntitas);
        if (data.isPresent()) {
            return new DataResponse < > (true, ResponseMessage.DATA_FETCHED,
toPojo(data.get()));
        } else {
            throw new NotFoundException(ResponseMessage.DATA_NOT_FOUND);
        }
    } catch (Exception e) {
        log.error("Error when get detail entitas.", e);
        throw e;
    }
}
```

Unit Testing Menggunakan JUNIT + Mockito

```
@DisplayName("Test fungsi Data Not Found pada method findOne")
@Test
void test_findOne_Not_Found() throws Exception {
    when(entitasRepository.findById(idEntitas)).thenReturn(Optional.empty());

    NotFoundException e = assertThrows(NotFoundException.class,
        () -> {
            entitasService.findOne(idEntitas);
        });
    assertEquals(ResponseMessage.DATA_NOT_FOUND, e.getMessage());
}
```

Unit test diatas akan memverifikasi *method* `findOne(String idEntitas)` pada *Class* `EntitasServiceImpl`. Jika `entitasRepository.findById(idEntitas)` memberikan *return* kosong (data tidak tersedia) maka *method* diwajibkan untuk melakukan *Throw error* menggunakan

NotFoundException.class. Berikut adalah contoh code *method findOne(String idEntitas)* yang tidak lolos uji.

```
@Override
public DataResponse < EntitasPojo > findOne(String idEntitas) {
    try {
        Optional < TdEntitas > data = entitasRepository.findById(idEntitas);
        if (data.isPresent()) {
            return new DataResponse < > (true, ResponseMessage.DATA_FETCHED,
toPojo(data.get()));
        } else {
            return new DataResponse < > (true, ResponseMessage.DATA_FETCHED, new
EntitasPojo());
        }
    } catch (Exception e) {
        log.error("Error when get detail entitas.", e);
        throw e;
    }
}
```

2. Validation

Memastikan setiap proses input atau ubah data tervalidasi dengan baik sangat penting untuk menjaga kualitas perangkat lunak. Berikut adalah contoh *Unit Test* pada *Controller*.

Method *create* pada Class *EntitasController.java*

```
@PostMapping(produces = MediaType.APPLICATION_JSON_VALUE)
@ApiOperation("Tambah Data Entitas Baru")
public ResponseEntity < DataResponse < EntitasPojo >> create(@Valid
@RequestBody EntitasPojo entitas) {
    DataResponse < EntitasPojo > data = entitasService.create(entitas);
    return ResponseEntity.ok().body(data);
}
```

Unit Testing Menggunakan JUNIT + Mockito

```
@Test
void create_With_Invalid_Request_Body() throws Exception {
    String dataInJson = "{\"nibEntitas\":\"23232323121\",\"namaEntitas\":\"PT
Abc\"}";
    MvcResult mvcResult = mockMvc.perform(post(END_POINT)
        .content(dataInJson)
        .header(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON))
        .andExpect(status().isUnprocessableEntity())
        .andReturn();

    String responseBody = mvcResult.getResponse().getContentAsString();

    assertNotNull(responseBody);

    ErrorsResponse resp =
objectMapper.readValue(responseBody.getBytes(StandardCharsets.UTF_8),
ErrorsResponse.class);
    assertNotNull(resp);
}
```

```
assertFalse(resp.isStatus());
assertTrue(resp.getErrors().stream().anyMatch((a) ->
a.toLowerCase().contains("id header")));
assertTrue(resp.getErrors().stream().anyMatch((a) ->
a.toLowerCase().contains("seri entitas")));

verify(entitasService, times(0)).create(any(EntitasPojo.class));
}
```

Unit test diatas dibuat untuk memastikan *end point* POST /entitas yang didefinisikan pada *Controller* dapat menangani *Request Body* yang tidak sesuai spesifikasi yang telah ditentukan dan memberikan *response* yang sesuai.

Selain *response* yang memberikan informasi letak kesalahannya, *Unit Test* pada contoh diatas dibuat untuk memastikan tidak ada eksekusi *method* create() pada *entitasService*. Ini bertujuan untuk memastikan code ditulis secara optimal.